

שלום לכולם.

המשימה הזאת ממשיכה את העבודה על הפרוייקט המתגלגל שאיתו נסיים את הקורס. למי שמעוניינת, ניתן לעבוד בזוגות על הפרוייקט, מעכשיו ועד הסוף. אפשר גם לעבוד ביחידים. המשימה מערבת שימוש בכל הכלים שנגענו והשתמשנו בהם עד עכשיו – חלקם בצורה מעמיקה יותר וחלקם פחות.

במשימה הזו אנחנו לוקחים את האפליקציה שפרסנו ב AWS במשימה הראשונה ופורסים אותה בעזרת כלי IaC . אופן ההגשה של המשימה מוגדר במסמך. המשימה היא ללא ציון, אך חובת הגשה תקפה על מנת שנוכל לסיים את הקורס. אני תמיד זמין לשאלות או הערות, ואם מעוניינים, אפנה גם זמן כדי לתמוך בכם בביצוע.

פרויקט מתגלגל DEVOPS ON AWS

משימה 2

Terraform Infrastructure as Code & Configuration Management
Ansible +

המשימה מבוססת על המשימה הראשונה שבה הוקמה ידנית אפליקציה עם 3 שירותים על EC2 ושימוש ב־RDS, S3, nginx ו־SNS.

רקע

במשימה הראשונה הקמתם סביבת AWS בסיסית עבור אפליקציה שמורכבת מ-3 שירותים נפרדים, כאשר כל שירות רץ על EC2 נפרד. בנוסף השתמשו בשירותים כמו S3, RDS, SNS ו-nginx, ותיעדתם את הארכיטקטורה והחיבורים בין הרכיבים.

במשימה הזו ניקח את אותו פרויקט ונעביר אותו לשלב מקצועי יותר בעולם ה-DevOps:

- במקום להקים תשתיות ידנית דרך AWS Console, התשתית תוגדר ותיפרס באמצעות Terraform.

- במקום להתחבר לשרתים ידנית ולהתקין nginx, חבילות, קבצי קונפיגורציה ושירותים, פעולות ההתקנה והקונפיגורציה יתבצעו באמצעות Ansible.

מטרת המשימה

מטרת המשימה היא לתרגל עבודה עם שני כלים מרכזיים בעולם ה-DevOps:

1. Terraform - לניהול והקמה של תשתיות בענן כקוד.

2. Ansible - לניהול קונפיגורציה, התקנות ופריסת אפליקציה על שרתים.

בסיום המשימה, כל סביבת הפרויקט שלכם אמורה להיות ניתנת להקמה מחדש בצורה אוטומטית, מסודרת ומתועדת.

תיאור המשימה

עליכם לקחת את הסביבה שבניתם במשימה הראשונה, ולבנות לה תהליך הקמה אוטומטי.

כל רכיבי ה Infrastructure שנוצרו ידנית ב-AWS צריכים להיווצר באמצעות Terraform. כל הפעולות שביצעתם ידנית בתוך השרתים צריכות להתבצע באמצעות Ansible.

- יצירת EC2 instances

- יצירת Security Groups

- יצירת RDS

- יצירת S3 Bucket

- יצירת SNS Topic ו-Subscription

- התקנת packages וכלים על סוגי השרתים השונים – web, backend

- התקנת runtime לשירותים, לדוגמה Python
- העתקת קבצי האפליקציה לשרתים.
- יצירת קבצי config או environment variables
- הפעלה או restart של השירותים

דרישות Terraform

1. מבנה פרויקט Terraform

יש ליצור תיקייה ייעודית עבור קוד ה-Terraform, כמו שבנינו בשיעורים

2. משאבי AWS נדרשים

Networking

VPC •

Public Subnet עבור השרת החשוף החוצה.

Private Subnets עבור רכיבים פנימיים, אם בחרתם להשתמש בהם.

Internet Gateway במידת הצורך.

Route Tables •

Security Groups עם הרשאות מינימליות.

EC2

יש להקים 3 שרתי EC2:

שרת Frontend / nginx •

שרת Backend •

שרת Service / Worker נוסף •

דרישות:

לכל שרת יהיה שם ברור ושימוש ב tags.

יש להגדיר Security Groups מתאימים לכל שרת.

יש להגדיר key pair או מנגנון גישה אחר בצורה מתועדת.

- יש לוודא ש Ansible יוכל להתחבר לשרתים לאחר יצירתם.

RDS

- יש להקים RDS עבור בסיס הנתונים של האפליקציה.
- יש להשתמש PostgreSQL.
- יש להגדיר subnet group במידת הצורך.
- Security Group צריך לאפשר גישה רק מהשרתים הרלוונטיים.

S3

- יש להקים Bucket עבור שמירת קבצים.
- שם ה bucket יוגדר דרך variable או convention ברור.

SNS

- יש להקים SNS Topic.
- יש להקים Email Subscription.
- יש להגדיר הרשאות נדרשות לשליחת הודעה מהאפליקציה.

3. Variables ו-Outputs

יש להשתמש ב-variables עבור ערכים משתנים, לדוגמה:

• AWS region

• instance type

• AMI ID

יש להגדיר outputs שימושיים, לדוגמה:

• Public IP של שרת ה nginx

• Private IPs של שאר השרתים

4. State

יש להסביר ב README איך מנוהל ה-Terraform state.

- איפה נמצא קובץ ה state, ומדוע

דרישות Ansible

1. מבנה פרויקט Ansible

יש ליצור תיקיה ייעודית, ובמבנה הנכון, כמו שנלמד בשיעורים.

על הקוד להיות קריא, מסודר ומתועד.

2. Inventory

יש ליצור inventory שמאפשר ל Ansible לדעת אילו שרתים שייכים לאיזה תפקיד.

בנוסף: יצירת inventory דינמי מתוך outputs של Terraform.

3. התקנות בסיסיות

יש ליצור playbook שמבצע התקנות בסיסיות על כל השרתים בפרוייקט, לדוגמה:

- עדכון package manager.
- התקנת חבילות בסיסיות.
- יצירת תיקיות נדרשות.
- יצירת משתמש ייעודי להרצת האפליקציה, אם רלוונטי.

4. התקנת nginx וקונפיגורציה

על Ansible להתקין ולהגדיר nginx בשרת ה Frontend.

- התקנת nginx.
- יצירת קובץ config מתוך template.
- הגדרת reverse proxy לשירות הרלוונטי, אם נדרש.

5. פריסת שירותי האפליקציה

על Ansible לפרוס את שלושת השירותים. לכל שירות יש להגדיר:

- העתקת קוד או artifact לשרת.
- התקנת dependencies.
- יצירת קובץ configuration או environment.
- הגדרת systemd service או דרך אחרת להרצת השירות.

6. קונפיגורציה מול AWS Services

יש לוודא שהאפליקציה מקבלת את הפרטים הנדרשים כדי לעבוד מול:

- RDS endpoint
- DB username/password
- S3 bucket name
- SNS topic ARN
- region

דרישות קוד

- הקוד חייב להיות קריא ומסודר.
- יש להשתמש בשמות משתנים ברורים.
- יש להוסיף comments משמעותיים בקוד.
- אין צורך להגיב על כל שורה, אבל כן צריך להסביר חלקים חשובים.
- יש להימנע מערכים hard-coded כאשר מתאים להשתמש ב variables.
- אין להעלות secrets, private keys או passwords ל-Git.
- יש לצרף קובצי example במקום קבצים שמכילים מידע רגיש.

תוצרים נדרשים

1. קוד Terraform

יש להגיש את כל קוד ה-Terraform שכתבתם. הקוד צריך לאפשר הרצה של:

```
terraform init  
terraform plan  
terraform apply
```

בנוסף, יש לתעד איך להריץ:

```
terraform destroy
```

2. קוד Ansible

יש להגיש את כל קוד ה-Ansible שכתבתם. הקוד צריך לאפשר הרצה של playbook שמגדיר את השרתים ומפעיל את האפליקציה.

```
ansible-playbook -i inventory.ini playbook.yml
```

3. README מסודר

יש לצרף README שמסביר:

- מה הארכיטקטורה של הפרויקט.
- אילו רכיבים נוצרים על ידי Terraform.
- אילו פעולות מתבצעות על ידי Ansible.
- איך להריץ את Terraform.
- איך להריץ את Ansible.
- אילו variables צריך להגדיר לכל כלי.
- אופן הטיפול והניהול של secrets.
- איך בודקים שהמערכת עובדת.
- איך מוחקים את הסביבה בסיום.

- **בנוסף:** אילו **בעיות** נתקלתם בהן ואיך פתרתם אותן.

4. **תרשים ארכיטקטורה מעודכן**

יש לצרף תרשים שמציג את הסביבה כפי שהיא מוקמת אוטומטית. התרשים צריך לכלול:

VPC / Subnets •

EC2 instances •

Security Groups •

RDS •

S3 •

SNS •

nginx •

• כיווני תקשורת בין הרכיבים

• הבחנה בין רכיבים שנוצרים על ידי Terraform לבין פעולות שמבוצעות על ידי Ansible

בדיקות נדרשות

יש להראות שהמערכת עובדת לאחר הקמה אוטומטית. יש לצרף צילומי מסך או פלטים מהפקודות הבאות:

```
terraform plan
terraform apply
terraform output
ansible-playbook -i inventory.ini playbook.yml
```

בנוסף, יש להראות:

• גישה לאפליקציה דרך HTTP.

דגשים טכניים

• אין לבצע ידנית פעולות שאמורות להיות אוטומטיות.

- אם נאלצתם לבצע פעולה ידנית, יש לתעד אותה ולהסביר מדוע.
- יש להקפיד על הרשאות IAM ו Security Groups מינימליות.
- יש להימנע מפתחת פורטים לא נחוצים.
- יש להימנע משימוש ב-0/0.0.0.0 למעט HTTP/HTTPS לשרת הציבורי, ואם יש צורך גם SSH - להסביר למה.
- יש לתעד כל החלטה ארכיטקטונית משמעותית.

בונוס

- יצירת inventory דינמי ל Ansible מתוך Terraform outputs.
- שימוש ב Ansible Vault לניהול secrets.
- הגדרת HTTPS עם certificate.
- הוספת health checks לשירותים.
- יצירת script אחד שמריץ את כל תהליך ההקמה לפי סדר.

אופן ההגשה

יש להגיש במייל מסודר את התוצרים הבאים:

1. קישור ל Git repository או קובץ ZIP עם הקוד.
2. README מלא.
3. תרשים ארכיטקטורה.
4. צילומי מסך או פלטי פקודות שמוכיחים שהמערכת עובדת.
5. הסבר קצר על חלוקת העבודה, אם עבדתם בזוגות.

לוח זמנים מומלץ

זמן מומלץ לביצוע: 15 - 12 שעות.

המשימה מעט יותר מורכבת מהמשימה הראשונה, משום שהיא דורשת לחשוב לא רק איך מקימים את המערכת, אלא איך הופכים את ההקמה שלה לתהליך אוטומטי, חוזר, קריא וניתן לתחזוקה.

על מה לשים דגש?

- האם אפשר להקים את הסביבה מחדש מאפס.
- האם הקוד ברור ומחולק נכון.
- האם יש comments משמעותיים.
- האם יש הפרדה נכונה בין Terraform ל Ansible.
- האם ה Security Groups והרשאות IAM מוגדרים בצורה מינימלית.
- האם התייעוד מספיק ברור כדי שמישהו אחר יוכל להריץ את הפרויקט.
- האם את'ה מבינ'ה את הקוד שנכתב ויכול'ה להסביר אותו.